WEB DESIGN · CSS

CSS

Cascading Style Sheets is the language that controls how a web page looks: colors, fonts, spacing, layout and motion. This app is written in HTML and starts with no CSS of its own. Each chapter adds one rule to its style sheet, so the page changes as the lesson goes on. Move with the arrow keys or the buttons at the bottom.

What this lesson covers

1. How a CSS rule is written and where it lives
2. Colors, backgrounds, units and typography
3. Selectors: targeting exactly the elements you want
4. Specificity, the cascade and inheritance
5. The box model, display types, borders and shadows
6. Layout with flexbox and grid
7. Responsive design with media queries
8. Transitions and animations
9. A project: style a profile card

How to use it

The **style.css** button at the top shows the rules the app is using right now. Going forward adds a rule. Going back removes it.

Most chapters have a demo on the right. Everything in the demos is live CSS: change a value and the browser redraws it immediately.

STRUCTURE AND PRESENTATION

Two languages, two jobs

HTML describes what each piece of content is. CSS describes how it should look. Keeping them separate means one style sheet can restyle an entire website without touching its content.

HTML	Structure and meaning: this is a heading, this is a paragraph, this is a button.
CSS	Presentation: color, size, font, spacing, position and motion.
Browser	Reads both and draws the result. Without CSS it uses its own default styles, which is why unstyled pages all look alike.

The card on the right uses the same HTML in both states. Only the CSS is switched.

Turn CSS on

```
<div class="card">
  <h3>Alex Morgan</h3>
  <p>Front-end developer in Sofia.</p>
  <button>Follow</button>
</div>
```

RENDERED RESULT

Alex Morgan

Front-end developer in Sofia.

Follow

CONNECTING CSS TO HTML

Three places to write CSS

CSS can sit on a single element, inside the page, or in a separate file. Real websites almost always use a separate file, because many pages can share it.

Inline | Internal | **External**

```
<!-- index.html -->
<head>
  <link rel="stylesheet" href="style.css">
</head>

/* style.css */
p {
  color: crimson;
}
```

External style sheet

A separate .css file, connected with a <link> element in the <head>.

GOOD FOR

Real websites. One file styles every page, and the browser downloads it once and keeps it in its cache.

DRAWBACK

One more file to manage. This app works this way: open style.css at the top.

SYNTAX

Anatomy of a rule

Every CSS rule has the same shape: a selector, then a declaration block in curly braces. Select each highlighted part to see its name.

```
h1 {
  color: crimson;
  font-size: 32px;
}
/* main heading */
```

> Selector: which elements the rule applies to. Here, every <h1> element.

Build a rule

Selector: h1

Property: color

Value: crimson

```
h1 {
  color: crimson;
}
```

Page title

The first paragraph.

> A paragraph with class="note".

Button

SYNTAX

Spot the mistake

When CSS has a syntax error, the browser skips the broken part without any warning. Each rule below has one error. Find it, then reveal the answer.

1. Show answer

```
h1 {
  color: crimson
  font-size: 32px;
}
```

2. Show answer

```
p {
  color = blue;
}
```

3. Show answer

```
note {
  color: gray;
}
```

4. Show answer

```
h2 {
  colour: navy;
}
```

5. Show answer

```
p {
  font-size: 18;
}
```

6. Show answer

```
button {
  padding: 10 px;
}
```

7. Show answer

```
.card {
  border: 1px solid black;
```

8. Show answer

```
#Menu {
  display: flex;
}
```

PROPERTY · COLOR

Color

The `color` property sets the text color. The same color can be written in several formats. The color you choose here becomes the heading color of this app.

Name Hex rgb() hsl()



About 140 color names are built in, such as crimson, teal and rebeccapurple.

```
h1, h2, h3 {
  color: #D61F69;
}
```

Heading text

Body text stays dark so that it is easy to read.

Contrast on white: 4.9 : 1. Readable at any size. The app headings now use this color.

Contrast ratio compares the lightness of text and background. Accessibility guidelines ask for at least 4.5 : 1 for normal text and 3 : 1 for large headings.

STYLE.CSS GETS `h1, h2, h3 { color: #d61f69; }`

PROPERTY · BACKGROUND

Background

`color` paints the text; `background` paints the area behind it. A background can be a single color or a gradient that blends several colors. A light solid color you pick here becomes this app's background.

Solid linear-gradient radial-gradient

Custom

```
body {
  background: #F1F0FF;
}
```

The app background now uses this color.



STYLE.CSS GETS `body { background: #f1f0ff; }`

VALUES · UNITS

Units: px, %, em, rem, vw

Sizes need a unit. Absolute units are always the same size. Relative units depend on something else, such as the parent element, the font size or the window, which is what lets a page adapt to different devices. Pick a unit and move the sliders.

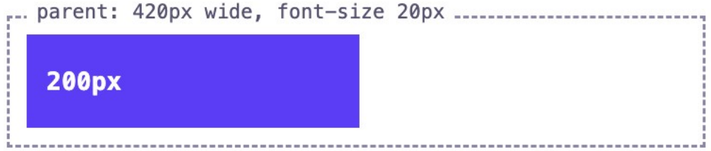
UNIT	RELATIVE TO	TYPICAL USE
`px`	Nothing: a fixed size	Borders, shadows
`%`	The parent's size	Widths
`em`	The element's font size	Padding around text
`rem`	The root (html) font size	Font sizes, spacing
`vw`	1% of the window width	Full-width sections

width 10em

parent width 420px

font-size 20px

root font-size 16px



`10em × 20px font-size = 200px`

```
html {
  font-size: 16px;
}
.parent {
  width: 420px;
  font-size: 20px;
}
.bar {
  width: 10em;
}
```

Browsers start with a root font size of 16px, so `1rem` is usually 16px. Using rem for font sizes respects readers who set a larger default text size in their browser.

PROPERTIES · FONT AND TEXT

Typography

Most of a web page is text, so typography matters more than any other property. `font-family` takes a list of fonts; the browser uses the first one that is installed. The font you pick becomes this app's heading font.

FONT-FAMILY

Bricolage Grotesque | Instrument Sans | Georgia | Times New Roman | Arial | Verdana | Trebuchet MS | Courier New

font-size 20px

font-weight 400

line-height 1.5

letter-spacing 0px

TEXT-ALIGN **left** | center | right | justify

TEXT-TRANSFORM **none** | uppercase | capitalize

```
p {
  font-family: 'Bricolage Grotesque', sans-serif;
  font-size: 20px;
  font-weight: 400;
  line-height: 1.5;
  letter-spacing: 0px;
  text-align: left;
  text-transform: none;
}
```

Reading on screens

Line height and letter spacing change how comfortable a paragraph is to read. Most body text reads well with a line height between 1.4 and 1.7 and a line length of about 60 to 75 characters.

End every font list with a generic family: `serif`, `sans-serif` or `monospace`. If no listed font is installed, the browser still picks something sensible.

STYLE.CSS GETS `body { font-family: 'Instrument Sans', sans-serif; } h1, h2, h3 { font-family: 'Bricolage Grotesque', sans-serif; } h2 { font-size: 2.9rem; font-weight: 750; }`

SELECTORS · 1 OF 6

Basic selectors

A selector decides which elements a rule applies to. These four are the foundation. Select a row or type your own selector: every element it matches lights up in the page source on the right.

SELECTOR	NAME	MATCHES
`p`	Type	Every <p> element
`.note`	Class	Every element with `class="note"`. Many elements can share a class.
`#top`	ID	The one element with `id="top"`. An id must be unique on the page.
`*`	Universal	Every element

YOUR SELECTOR

p

4 elements match.

> An element can have several classes separated by spaces: `class="new featured"` matches both `.new` and `.featured`.

PAGE SOURCE

```
<header id="top">
  <h1>City Library</h1>
  <nav>
    <a href="/">Home</a>
    <a href="/news" class="active">News</a>
    <a href="https://events.example.org">Events</a>
  </nav>
</header>
<article>
  <p class="intro">The new wing opens on Monday.</p>
  <p>It adds three rooms.</p>
  <ul>
    <li>Study rooms</li>
    <li class="new featured">Maker lab</li>
    <li class="new">Media archive</li>
  </ul>
  <p class="note">Entry is free.</p>
  <a href="/files/plan.pdf">Floor plan</a>
</article>
<footer>
  <p class="note">Open 9:00–20:00</p>
</footer>
```

STYLE.CSS GETS `.tag { background: #d61f69; color: white; }`

SELECTORS · 2 OF 6

Grouping and combining

Selectors can be joined. A comma makes a list of separate selectors. Two selectors written with no space between them describe one element that must match both. A space means something different, covered in the next chapter.

SELECTOR	MEANING
`h1, footer p`	Group: the h1 **and** the paragraph in the footer. The same as writing two rules.
`li.new`	Compound: an `li` that also has class `new`
`p.intro`	Compound: a paragraph with class `intro`
`.new.featured`	Compound: an element that has **both** classes
`a.active`	Compound: the link with class `active`

YOUR SELECTOR

```
li.new
```

2 elements match.

> `p.intro` (no space) is a paragraph with class intro. `p .intro` (with a space) is an element with class intro *inside* a paragraph. One space changes the meaning.

PAGE SOURCE

```
<header id="top">
  <h1>City Library</h1>
  <nav>
    <a href="/">Home</a>
    <a href="/news" class="active">News</a>
    <a href="https://events.example.org">Events</a>
  </nav>
</header>
<article>
  <p class="intro">The new wing opens on Monday.</p>
  <p>It adds three rooms.</p>
  <ul>
    <li>Study rooms</li>
    <li class="new featured">Maker lab</li>
    <li class="new">Media archive</li>
  </ul>
  <p class="note">Entry is free.</p>
  <a href="/files/plan.pdf">Floor plan</a>
</article>
<footer>
  <p class="note">Open 9:00–20:00</p>
</footer>
```

SELECTORS · 3 OF 6

Combinators: selecting by position

Combinators select elements by their place in the HTML tree. Look at the indentation in the source: an element inside another is its descendant, an element directly inside is its child, and elements next to each other are siblings.

SELECTOR	NAME	MATCHES
`article p`	Descendant (space)	Every p anywhere inside article, at any depth
`header > nav`	Child (>)	A nav directly inside header, one level down only
`header > a`	Child (>)	Nothing: the links are inside nav, not directly inside header
`h1 + nav`	Next sibling (+)	A nav immediately after an h1, with the same parent
`li + li`	Next sibling (+)	Every li that follows another li: all except the first
`.intro ~ p`	Later siblings (~)	Every p after .intro with the same parent, not only the next one

YOUR SELECTOR

```
article p
```

3 elements match.

PAGE SOURCE

```
<header id="top">
  <h1>City Library</h1>
  <nav>
    <a href="/">Home</a>
    <a href="/news" class="active">News</a>
    <a href="https://events.example.org">Events</a>
  </nav>
</header>
<article>
  <p class="intro">The new wing opens on Monday.</p>
  <p>It adds three rooms.</p>
  <ul>
    <li>Study rooms</li>
    <li class="new featured">Maker lab</li>
    <li class="new">Media archive</li>
  </ul>
  <p class="note">Entry is free.</p>
  <a href="/files/plan.pdf">Floor plan</a>
</article>
<footer>
  <p class="note">Open 9:00–20:00</p>
</footer>
```

SELECTORS · 4 OF 6

Attribute selectors

Square brackets select elements by their attributes and attribute values. They are useful for links, form fields and anything without a class.

SELECTOR	MATCHES
`[required]`	Has the attribute, whatever its value
`input[type="email"]`	Value is exactly `email`
`a[href^="https"]`	Value **starts** with https: links to other sites
`a[href$=".pdf"]`	Value **ends** with .pdf
`a[href*="library"]`	Value **contains** library anywhere
`button[type="submit"]`	The submit button only

YOUR SELECTOR

```
[required]
```

2 elements match.

> Memory aid: ^ marks the start and $ the end, as in many programming tools. * means anywhere.

PAGE SOURCE

```
<form>
  <label for="name">Name</label>
  <input type="text" id="name" required>
  <label for="mail">Email</label>
  <input type="email" id="mail" required>
  <input type="checkbox" id="news" checked>
  <button type="submit">Sign up</button>
  <button type="reset" disabled>Clear</button>
</form>
<a href="/library/rules">Rules</a>
<a href="/files/plan.pdf">Floor plan</a>
<a href="https://example.org">Partner site</a>
```

SELECTORS · 5 OF 6

Pseudo-classes: state and position

A pseudo-class starts with one colon. It selects an element while it is in a certain state, such as being hovered, or because of its position among its siblings.

State

Turn rules on, then use the form below. These states exist only while you interact with the page.

	RULE
☑	`button:hover { background: #5B3DF5; color: white }`
☑	`input:focus { outline: 3px solid #5B3DF5; background: #F1F0FF }`
☑	`input:checked + span { font-weight: bold; color: #0B7A4B }`
☑	`button:disabled { opacity: .4; cursor: not-allowed }`
☐	`input:placeholder-shown { border-style: dashed }`

Click here to focus

☑ Subscribe to updates

Hover me | Disabled

Position

LI: `nth-child(odd)`

`:first-child` `:last-child` `:nth-child(3)` **`:nth-child(odd)`**
`:nth-child(even)` `:nth-child(3n)` `:nth-child(3n+1)` `:nth-child(-n+4)`
`:not(:nth-child(5))`

li:nth-child(odd) matches items 1, 3, 5, 7, 9, 11.

1	2	3	4	5	6
7	8	9	10	11	12

`nth-child(an+b)`: n counts 0, 1, 2, 3… For `3n+1` that gives 1, 4, 7, 10. `odd` and `even` are shortcuts for `2n+1` and `2n`.

SELECTORS · 6 OF 6

Pseudo-elements: styling part of an element

A pseudo-element starts with two colons. It styles a part of an element that has no tag of its own, such as the first letter, or adds generated content before or after it. Turn each rule on to see what it does.

	PSEUDO-ELEMENT	TARGETS
☑	`::first-letter`	the first letter of a block
☐	`::first-line`	the first rendered line
☑	`::before`	new content before the element
☐	`::after`	new content after the element
☐	`::marker`	the bullet of a list item
☐	`::placeholder`	the hint text in an empty field
☑	`::selection`	text the user selects

::before text →

```
p::first-letter { font-size: 2.6em; float: left; line-height: 1;
margin-right: 6px; color: #D61F69; }
li::before { content: "→ "; color: #5B3DF5; }
::selection { background: #FFE14D; color: #17152E; }
```

Opening times

The library is open every day of the week. Opening hours change on public holidays, so check the calendar before you visit. Select some of this text to see ::selection.

- → Monday to Friday 9:00 to 20:00
- → Saturday 10:00 to 18:00
- → Sunday 12:00 to 17:00

Search the catalogue

Pseudo-classes (one colon) select whole elements in a state or position. Pseudo-elements (two colons) target a piece of an element.

PRACTICE

Selector practice

Write one selector for each task. A task is complete when your selector matches exactly the right elements and nothing else. Select a task to work on it.

1. Every link inside the navigation
2. Only the link with class active
3. List items that have class new
4. Paragraphs inside the article, not the footer
5. The link to the PDF file
6. The first list item only
7. Every list item except the first
8. The link that leaves the site
9. Elements directly inside the header
10. Every paragraph after the intro paragraph
11. Both elements with class note
12. The heading and the footer paragraph together

TASK 1: EVERY LINK INSIDE THE NAVIGATION

Type a selector.

Show a hint

PAGE SOURCE · 0 OF 12 TASKS COMPLETE

```
<header id="top">
  <h1>City Library</h1>
  <nav>
    <a href="/">Home</a>
    <a href="/news" class="active">News</a>
    <a href="https://events.example.org">Events</a>
  </nav>
</header>
<article>
  <p class="intro">The new wing opens on Monday.</p>
  <p>It adds three rooms.</p>
  <ul>
    <li>Study rooms</li>
    <li class="new featured">Maker lab</li>
    <li class="new">Media archive</li>
  </ul>
  <p class="note">Entry is free.</p>
  <a href="/files/plan.pdf">Floor plan</a>
</article>
<footer>
  <p class="note">Open 9:00–20:00</p>
</footer>
```

SPECIFICITY

Specificity: how exact is a selector?

When two rules set the same property on the same element, the browser uses the more specific selector. Specificity is counted in three columns, ids, classes and types, and compared from the left: one id beats any number of classes.

TYPE A SELECTOR

`nav .menu a`

0	1	2
IDs #name	Classes, attributes, pseudo-classes .x [type] :hover	Types, pseudo-elements p a ::before

Specificity 0-1-2.

`p` `.cold` `#v` `p.cold` `nav .menu a` `#v.cold` `a:hover`

`ul li::before` `input[type="text"]` `:not(#v) p`

Compare two selectors

`#v` **vs** `p.cold.big`

#v wins, 1-0-0 against 0-2-1. The IDs column decides.

SELECTOR	SCORE	WHY
`p`	`0-0-1`	one type
`.cold`	`0-1-0`	one class
`p.cold:hover`	`0-2-1`	class and pseudo-class, one type
`#v`	`1-0-0`	one id
`style="…"`	above all	inline style on the element

STYLE.CSS GETS `#next { background: #d61f69; font-weight: 700; }`

THE CASCADE

When rules conflict

The "cascading" in CSS is the set of steps the browser follows when several rules compete for the same property. It checks `!important` first, then specificity, then source order: if everything else is equal, the rule written last wins.

STYLE.CSS, TOP TO BOTTOM

☑	`p { color: royalblue; }`	0-0-1	!	↑	↓
☑	**`.cold { color: seagreen; }`**	0-1-0 **applies**	!	↑	↓
☑	`p { color: crimson; }`	0-0-1	!	↑	↓
☐	~~`#v { color: darkorange; }`~~	1-0-0	!	↑	↓
☐	~~`.cold.big { color: mediumvioletred; }`~~	0-2-0	!	↑	↓
☐	~~`p.cold { color: teal; }`~~	0-1-1	!	↑	↓

Turn rules on and off, mark one `!important`, and move rules up and down. Predict the result before it changes.

`<P CLASS="COLD BIG" ID="V">`

Vanilla

.cold { color: seagreen } applies. It has the highest specificity, 0-1-0.

1. `!important` beats every normal declaration.
2. Higher specificity beats lower.
3. On a tie, the later rule wins.

Use !important rarely. It breaks the normal order and makes later changes harder. A more specific selector is usually the better fix.

INHERITANCE

Inheritance

Some properties pass from a parent element to everything inside it. Text properties such as color and font are inherited. Box properties such as border and padding are not; otherwise every child would get its own border. Turn properties on for the parent and watch the children.

	ON .PARENT	CHILDREN
☑	`color: #5B3DF5`	inherited
☑	`font-family: Georgia, serif`	inherited
☐	`font-style: italic`	inherited
☐	`letter-spacing: 2px`	inherited
☑	`border: 3px solid #17152E`	not inherited
☑	`padding: 16px`	not inherited
☐	`background: #FFF4C2`	not inherited

☐ `button { font: inherit; color: inherit; }`

Form controls such as buttons do not take the parent's font by default. `inherit` makes them use it.

.parent has the rules on the left.

This paragraph has no rules of its own.

A nested div, two levels down.

A button

```
.parent {
  color: #5B3DF5;
  font-family: Georgia, serif;
  border: 3px solid #17152E;
  padding: 16px;
}
```

BOX MODEL

Every element is a box

Each element is a rectangle made of four layers: the content, padding around it, a border, and a margin outside the border that keeps other elements away. `box-sizing` decides whether width includes the padding and border.

width 160px

padding 20px

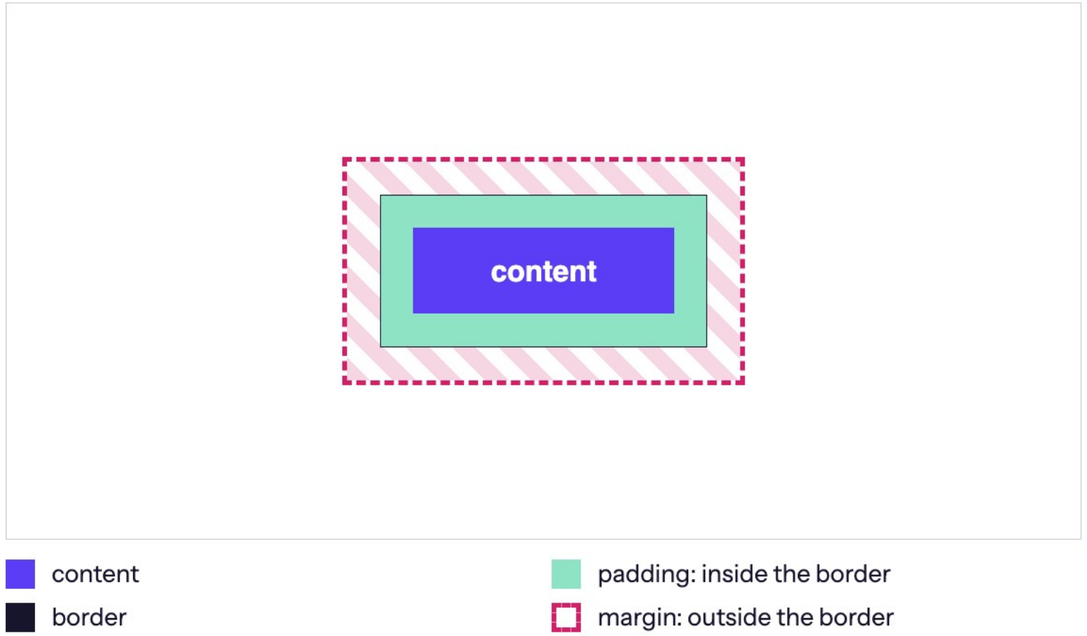
border 6px

margin 20px

BOX-SIZING **content-box** border-box

`Visible width = 160 + 2×20 + 2×6 = 212px`

```
.box {
  box-sizing: content-box;
  width: 160px;
  padding: 20px;
  border: 6px solid;
  margin: 20px;
}
```



STYLE.CSS GETS `.panel { padding: 22px; border: 2px solid; }`

PROPERTY · DISPLAY

display: block, inline and inline-block

Every element has a display type that decides how it sits next to others. Paragraphs start on a new line; links flow inside the text. The `display` property changes this. Pick a value and compare the three boxes.

VALUE	BEHAVIOR
`block`	New line, full width. width and height work.
`inline`	Flows in the text. width and height are ignored.
`inline-block`	Flows in the text, but keeps its width and height.
`none`	Removed. Takes up no space.
`visibility: hidden`	For comparison: invisible, but keeps its space.

Block by default: div, p, h1–h6, ul, section. **Inline by default:** span, a, strong, em. **Inline-block:** button, input.

> `flex` and `grid` are display values too. They are covered in the layout chapters.




```
.item { display: inline; width: 120px; height: 44px; padding: 4px 12px; margin: 8px; }
```

The boxes sit inside the text. The width and height of 120px and 44px are ignored, and the dashed margins overlap the lines above and below.

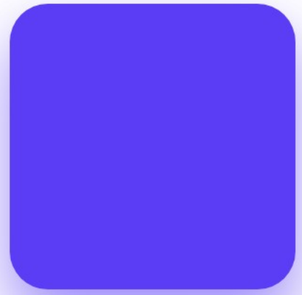
BORDER-RADIUS · BOX-SHADOW

Corners and shadows

`border-radius` rounds the corners of a box; 50% on a square gives a circle. `box-shadow` takes an x offset, a y offset, a blur, a spread and a color.

Card | Pill | Circle | Offset | Glow | None

radius		24px
x offset		0px
y offset		20px
blur		40px
spread		0px
shadow color		



```
.card {
  border-radius: 24px;
  box-shadow: 0px 20px 40px 0px #5b3df599;
}
```

STYLE.CSS GETS `.panel { border-radius: 18px; box-shadow: 0 22px 44px -26px #0007; }`

LAYOUT · FLEXBOX

Flexbox

`display: flex` goes on the parent and lines its children up along one axis. `justify-content` places them along that axis and `align-items` across it. Match the dashed outlines to complete each level.

FLEX-DIRECTION

row | column

JUSTIFY-CONTENT

flex-start | center | flex-end | space-between | space-around | space-evenly

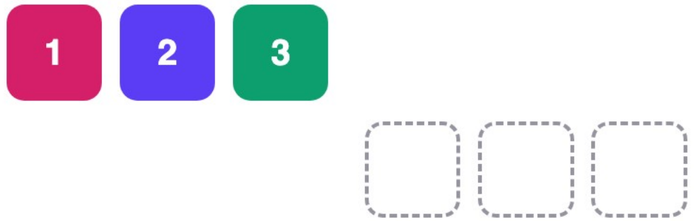
ALIGN-ITEMS

flex-start | center | flex-end

gap 12px

ITEMS − 3 +

LEVEL 1 OF 6: MATCH THE OUTLINES




```
.row {
  display: flex;
  flex-direction: row;
  justify-content: flex-start;
  align-items: flex-start;
  gap: 12px;
}
```

STYLE.CSS GETS `.bar, .navbar { display: flex; justify-content: space-between; }`

LAYOUT · GRID

Grid

Flexbox arranges items in one direction. Grid arranges them in rows and columns at the same time. `grid-template-columns` lists the column widths; the `fr` unit means one share of the free space.

GRID-TEMPLATE-COLUMNS

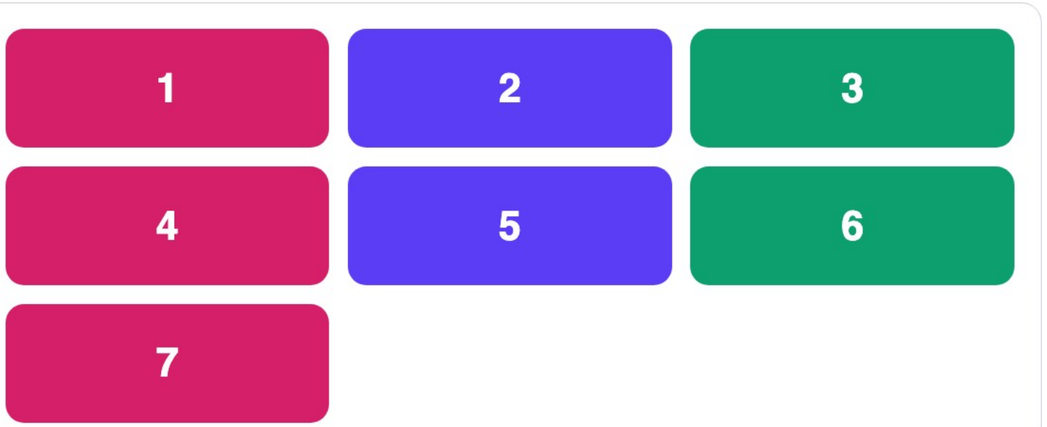
1fr 1fr | repeat(3, 1fr) | 200px 1fr | 1fr 2fr 1fr | repeat(4, 1fr)

gap 10px

items 7

☐ First item spans 2 columns

> Use flexbox for a row of buttons or a navigation bar. Use grid for a page layout or a gallery of cards.




```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
```

RESPONSIVE DESIGN · @MEDIA

Media queries

A media query applies rules only when a condition is true, most often the width of the screen. This is how one page adapts to a phone, a tablet and a desktop. Drag the width slider and watch which rules switch on.

screen width 1020px

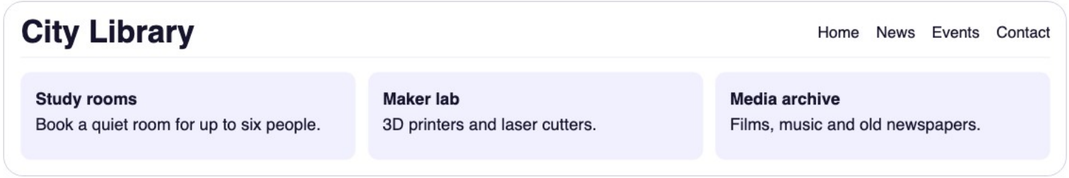
Phone 375px | Tablet 768px | Laptop 1280px

```
/* mobile first: always applies */
.cards { grid-template-columns: 1fr; }
nav { display: none; }

@media (min-width: 600px) {
  .cards { grid-template-columns: 1fr 1fr; }
  nav { display: flex; }
}

@media (min-width: 1000px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

> Write the phone layout first, then add `min-width` queries for larger screens. This is called mobile-first. A responsive page also needs `<meta name="viewport" content="width=device-width, initial-scale=1">` in its head.



Screen 1020px wide: both media queries apply, 3 columns.

Other media features: `prefers-color-scheme: dark`, `prefers-reduced-motion: reduce`, `orientation: portrait` and the `print` media type.

TRANSITION · ANIMATION

Transitions and animations

A transition animates the change between two states, for example when the pointer moves over a button. An animation runs on its own, following steps defined with `@keyframes`. This chapter also switches the app to a dark color scheme.

transition

duration 400ms

timing ease

grow spin lift color

Hover or tap

```
.btn { transition: all 400ms ease; }
.btn:hover { transform: scale(1.3); }
```

animation

bounce spin pulse shake wobble

duration 1000ms

ITERATIONS 1 3 infinite Play



```
@keyframes bounce {
  0%, 100% { transform: translateY(0); }
  50% { transform: translateY(-50px); }
}
.shape { animation: bounce 1000ms ease-in-out infinite; }
```

STYLE.CSS GETS `button { transition: transform .2s; } body { background: #100F22; color: #F2F0FF; }`

PROJECT

Style a profile card

Write CSS on the left; the card updates as you type. Each step button adds a starting rule for one topic. Change the values until the card is your own.

1·color 2·typography 3·box model 4·radius and shadow 5·layout 6·hover

STYLE.CSS

```
/* Write your CSS here. Start with the card itself: */
.card {
  background: white;
}
```

Copy CSS Start over

Name on the card Your Name



▶ Show the card's HTML

SUMMARY

Reference

Selectors

SELECTOR	MATCHES
`p .x #x *`	type, class, id, everything
`a, b`	a and b (group)
`a.x`	an a that also has class x
`a b`	b anywhere inside a
`a > b`	b directly inside a
`a + b` / `a ~ b`	b right after a / any b after a
`[attr]` `[attr="v"]`	has attribute / exact value
`^=` `$=` `*=`	starts with / ends with / contains
`:hover` `:focus` `:checked`	element in a state
`:first-child` `:nth-child()` `:not()`	position, or exclusion
`::before` `::first-letter`	generated content, part of an element

The cascade

`!important` → specificity (ids, classes, types) → source order.

Units

`px` fixed · `%` parent · `em` own font size · `rem` root font size · `vw` window width

display

`block` new line, full width · `inline` in the text, ignores width and height · `inline-block` in the text, keeps its size · `none` removed

Media queries

`@media (min-width: 600px) { … }` applies when the screen is at least 600px wide.

Properties in this lesson

`color` `background` `font-family` `font-size` `font-weight` `line-height` `letter-spacing` `text-align` `text-transform` `width` `padding` `border` `margin` `box-sizing` `display` `visibility` `border-radius` `box-shadow` `display` `flex-direction` `justify-content` `align-items` `gap` `grid-template-columns` `transition` `animation`

Further practice

MDN Learn CSS · CSS Diner · Flexbox Froggy · Grid Garden